

GMeet Minutes AI

AI-Generated Meeting Minutes for Google Meet

Full Project Plan

A web application that records a live Google Meet session, transcribes and summarizes it using the Gemini Flash API, stores meeting data in Supabase, and lets the host send the minutes by redirecting to Gmail with a pre-filled email.

Prepared for	BCA Final Year Project — BIT Mesra, Lalpur Extension
Team size	4 members
Core stack	React (Vite) + Supabase + Gemini Flash API
Document date	July 2026

1. Executive Summary

GMeet Minutes AI is a browser-based tool that removes the manual work of taking meeting notes. The host starts a recording from the web app while running a Google Meet call; the app captures the meeting audio, sends it for AI-based transcription and summarization, and produces a structured Minutes of Meeting (MoM) document — attendees, agenda, key discussion points, decisions, and action items. The host can then review the summary and send it to participants with one click, which opens Gmail with the subject and body already filled in.

The project reuses proven patterns from an earlier prototype (audio capture, AI summarization, Gmail redirect) and adds a proper Supabase-backed authentication and database layer so meetings, transcripts, and summaries persist per user instead of living only in the browser session.

2. Objectives

- Let a host record audio directly from an active Google Meet tab, no separate recording software.
- Automatically transcribe and summarize the meeting using the Gemini Flash API.
- Produce a structured MoM: title, date, attendees, agenda, discussion points, decisions, action items with owners.
- Persist users, meetings, and summaries in Supabase so history is available across sessions and devices.
- Let the host send the MoM by email through a one-click Gmail redirect, no manual copy-paste.
- Keep the whole pipeline usable within free-tier limits of Gemini Flash and Supabase, suitable for an academic demo.

3. Key Features

Core (must-have for evaluation)

- Email/password and Google OAuth login via Supabase Auth.
- In-tab audio capture of a Google Meet session using the browser's screen/tab-share API.
- Live recording indicator with start, pause, and stop controls.
- Audio sent to Gemini Flash for transcription + summarization in a single structured call.
- Auto-generated MoM: agenda, key points, decisions, action items (task + owner + due date if mentioned).
- Editable summary screen before sending — host can correct or add points.
- “Send via Gmail” button that opens Gmail compose pre-filled with subject, recipients, and formatted MoM body.
- Meeting history dashboard (past meetings, dates, quick re-open of any summary).

Stretch goals (nice-to-have, good for extra evaluation marks)

- Speaker labelling (“Speaker 1 / Speaker 2”) in the transcript.
- Direct send via Gmail API (OAuth) instead of just a compose redirect, so no manual click-to-send is needed.
- Shareable public link to a summary (read-only) instead of only email.
- Slack/Notion export of action items.
- Chrome extension version for background tab-audio capture without needing the screen-share picker.

4. Tech Stack

Layer	Technology	Purpose
Frontend	React + Vite, Tailwind CSS	SPA UI, fast dev/build, matches your existing stack from Saffron Sugar / MeetBrief.
Auth	Supabase Auth	Email/password + Google OAuth sign-in, session/JWT handling.
Database	Supabase Postgres	Stores users, meetings, transcripts, summaries, action items.
File storage	Supabase Storage	Stores raw audio recordings (optional, can auto-delete after processing).
AI engine	Gemini Flash API (multimodal)	Accepts the audio recording directly and returns transcript + structured MoM JSON in one call.
Email handoff	Gmail compose URL redirect	Opens gmail.com/mail with prefilled to/subject/body via query parameters — no backend mail server needed.
Hosting	Vercel / Netlify	Static frontend hosting, same flow you already used for the One-stack to-do app.

Note: Gemini Flash's multimodal input can accept audio files directly and return text, which means you do not need a separate speech-to-text service (like Whisper) — one API call can do transcription and summarization together. This is the main workflow difference from the earlier Claude-API version, where audio would first need to be transcribed by the browser's Web Speech API before summarization.

5. System Architecture & Data Flow

End-to-end flow from the moment the host presses “Start Recording” to the moment the MoM email is sent:

Step	What happens
1. Auth	Host logs into the web app via Supabase Auth (Google OAuth recommended, since it matches the Gmail identity used later).

Step	What happens
2. Setup	Host creates a “Meeting” record (title, expected attendees, agenda notes) — saved to Supabase before recording starts.
3. Capture	Host clicks “Start Recording”, then selects the Google Meet browser tab in the screen/tab-share picker with “Share tab audio” checked. The app records audio only (no video) using the MediaRecorder API.
4. Stop & upload	On “Stop”, the recorded audio blob is uploaded to Supabase Storage and linked to the meeting record.
5. AI processing	The app calls the Gemini Flash API with the audio file and a structured prompt requesting JSON output (transcript summary, decisions, action items).
6. Save results	The returned JSON is parsed and saved into the meetings/summaries tables in Supabase.
7. Review	Host is shown an editable MoM screen to fix names, action items, or wording.
8. Send	“Send via Gmail” builds a mailto:/Gmail-compose URL with the formatted MoM and opens it in a new tab, prefilled and ready to send.
9. History	The meeting and its MoM remain in the dashboard, searchable by title/date.

6. Supabase Database Schema

Suggested Postgres tables (with Row Level Security enabled so each user only sees their own data):

Table	Key columns	Notes
profiles	id (uuid, fk auth.users), full_name, email, avatar_url	Mirrors Supabase auth.users; created via a trigger on sign-up.
meetings	id, user_id (fk), title, meeting_date, status (recording/processing/done), audio_url	One row per recorded meeting.
summaries	id, meeting_id (fk), attendees[], agenda_text, key_points (jsonb), decisions (jsonb), raw_transcript	Structured AI output linked 1:1 with a meeting.
action_items	id, meeting_id (fk), description, owner_name, due_date, is_completed	One row per action item, lets you render a checklist.
email_logs	id, meeting_id (fk), sent_to[], sent_at	Optional audit trail of when/who the MoM was sent to.

Row Level Security policy pattern: every table filters on “auth.uid() = user_id” (directly, or via the parent meeting_id) so users can only read/write their own meetings.

7. Audio Capture: How It Actually Works in the Browser

A plain web app cannot silently listen to another browser tab. The reliable, permission-compliant approach is:

- Use `navigator.mediaDevices.getDisplayMedia({ video: true, audio: true })` — this opens Chrome's native “Share your screen” picker.
- Host picks the specific Google Meet browser tab and ticks “Share tab audio.”
- The app immediately stops/discards the video track and keeps only the audio track, feeding it into `MediaRecorder` to produce a `.webm/.wav` file.
- This means the host needs Chrome or Edge (tab-audio sharing is not available on all browsers/OS combinations — worth testing early and documenting as a known limitation in your report).
- Stretch goal: a Chrome extension using the `tabCapture` API removes the need for the picker each time, but that is extra scope beyond the base web app.

8. AI Summarization Pipeline (Gemini Flash)

Single Gemini Flash call with the audio file attached and a prompt instructing it to return only JSON, for example, asking for: meeting title, date, attendee names actually heard in audio, a plain-language summary, a list of key discussion points, a list of decisions made, and a list of action items each with a description, an owner (if mentioned), and a due date (if mentioned).

Keep the schema strict and always parse defensively (strip markdown code fences, wrap the parse in `try/catch`, and fall back to showing the raw text if JSON parsing fails) so a malformed model response never crashes the review screen.

9. Sending the Minutes via Gmail

Two implementation options, from simplest to most seamless:

Option	How it works	Effort
A. Gmail compose redirect	Build a URL to <code>https://mail.google.com/mail/?view=cm&fs=1&to;=...&su;=...&body;=...</code> with the recipients, subject, and formatted MoM text URL-encoded, then <code>window.open()</code> it. Gmail opens with everything prefilled; host clicks Send manually.	Low — no OAuth, no backend, works immediately.

Option	How it works	Effort
B. Gmail API send	Host connects their Google account via OAuth (Supabase supports Google as an auth provider, and the same token can be scoped for gmail.send). A Supabase Edge Function calls the Gmail API to send the email directly.	Higher — needs Google Cloud OAuth consent screen setup and scope approval; keep as a stretch goal.

Recommendation: ship Option A first since it fully satisfies the “redirect to Gmail” requirement with far less setup, then attempt Option B only if time remains.

10. Screens / UI Flow

- Landing / Login — split-layout page (reuse your MeetBrief design pattern) with Supabase email + Google sign-in.
- Dashboard — list of past meetings with status badges (Recording / Processing / Ready), a “New Meeting” button.
- Recorder screen — meeting title/agenda input, big Start/Stop control, live timer, capture-status indicator.
- Processing screen — simple loading state while audio uploads and Gemini Flash responds.
- Summary / Review screen — editable MoM sections (attendees, key points, decisions, action items) plus recipient input and the “Send via Gmail” button.
- Meeting detail (history) — read-only view of a past MoM, re-openable any time, with a “Resend” option.

11. Suggested Project Structure

```
gmeet-minutes-ai/  
  src/  
    components/      # Recorder, SummaryEditor, ActionItemList, MeetingCard  
    pages/           # Login, Dashboard, Recorder, Summary, MeetingDetail  
    lib/  
      supabaseClient.js  
      geminiClient.js # wraps the Gemini Flash API call + JSON parsing  
      gmailRedirect.js # builds the compose URL  
    hooks/  
      useAudioRecorder.js  
      useAuth.js  
    App.jsx  
    main.jsx  
  supabase/  
    schema.sql      # table + RLS definitions  
    functions/      # optional edge functions (Gmail API send, etc.)  
  .env.local        # VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY, VITE_GEMINI_API_KEY
```

12. Development Roadmap (4-person team, ~8 weeks)

Week	Focus	Suggested owner
1	Supabase project setup, schema + RLS, Auth screens (login/signup, Google OAuth).	Backend/DB person
2	React app scaffold, routing, dashboard UI shell, meeting CRUD against Supabase.	Frontend person

Week	Focus	Suggested owner
3	Audio capture: <code>getDisplayMedia</code> + <code>MediaRecorder</code> , upload to Supabase Storage.	Frontend + integration
4	Gemini Flash integration: prompt design, JSON schema, error handling.	AI integration person
5	Summary review/edit UI, action items list, save-back-to-DB.	Frontend person
6	Gmail redirect (Option A), recipient management, email formatting.	Full-stack pairing
7	Meeting history/detail views, polish, empty/error states, responsive layout.	Whole team
8	Testing, bug fixes, deployment (Vercel), documentation & report writing.	Whole team

13. Risks & Things to Watch

- Browser support: tab-audio sharing works reliably in Chrome/Edge but not all browsers — state this as a known limitation in your report rather than trying to support everything.
- Meeting consent: recording a call should only start after visibly notifying participants; mention this as a design consideration in your documentation (e.g. an on-screen recording indicator).
- API cost/limits: Gemini Flash has a free tier, but large audio files or many test runs can add up — cap recording length during development and testing.
- Audio quality: background noise or multiple people talking over each other will reduce transcription accuracy; note this as a limitation rather than trying to solve diarization in the base version.
- Gmail redirect depends on the host being signed into Gmail in that browser — acceptable for an academic project demo, but worth a one-line caveat in your report.

14. Future Enhancements (beyond the evaluation scope)

- Support Zoom and Microsoft Teams in addition to Google Meet.
- Real-time live captioning and a running summary during the call, not just after.
- Calendar integration (Google Calendar) to auto-create the meeting record from an event.
- Speaker diarization for accurate “who said what” attribution.
- Team workspaces so a MoM can be shared with a whole Supabase-managed organization, not just emailed out.

15. Conclusion

This plan keeps the base scope realistic for a four-person team within an academic timeline: a working record → transcribe → summarize → email pipeline, backed by real authentication and persistent storage in Supabase. The stretch goals give the team room to add polish or depth if the core pipeline is finished early, without putting the working demo at risk.